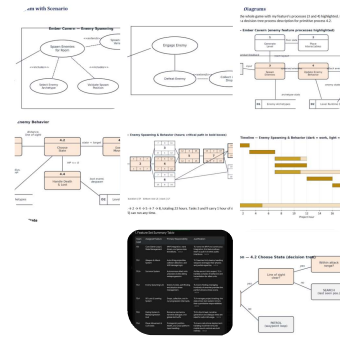


Today 12:30 PM



Champion_Template (1)(2...
Document

read these



I've read all of them carefully, including the **Champion Template** and the example diagrams/screenshots.

I understand that the screenshots are showing an example Champion Document mainly for **TL3 — Enemy Spawning & AI**, while the feature summary table shows your relevant feature as:

TL2+ — Summon System

Primary responsibility: **Autonomous allied units unlocked through the dating/metaprogression system.**

It involves **AI behavior and instantiation of allied units.**

The examples you gave include:

- **2 Use Case Diagrams**
 - System-side example: Level Generator → Spawn Enemies → Select Archetype / Validate Position / Elite Variant
 - Player-side example: Player → Engage Enemy → Defeat Enemy → Collect Loot
- **DFD Diagram 0** showing the whole game, with the feature processes highlighted.
- **Feature subprocess DFD**, such as 4.1 Sense Player → 4.2 Choose State → 4.3 Execute Movement → 4.4 Handle Death & Loot.
- **Primitive-process decision tree** for process 4.2.
- **PERT diagram** with ES, duration, EF, LS, slack, and LF.
- **Gantt timeline** separating actual work from slack.
- **Feature Set Summary Table** identifying responsibilities of TL1–TL6.

The actual Champion Template confirms that your final document needs **five substantive sections**: Introduction, Use Cases + scenarios, Data Flow Diagrams, Acceptance Tests, and

Timeline. It specifically requires **at least two UML use-case diagrams**, a whole-game Diagram 0 plus a feature subprocess diagram and primitive-process description, objective numerical acceptance tests, and a timeline containing a work-items table, PERT diagram, and Gantt chart. Champion_Template (1) +2

One important requirement I also noticed: your sections must be **internally consistent**—DFD processes need to correspond to the use cases, acceptance tests need to map to requirements, and timeline tasks need to match the work described. Champion_Template (1)

So I now understand the **exact visual style, structure, and level of detail** you want. For your **Summon System**, we can create equivalent original diagrams and tables—not copy the Enemy Spawning diagrams, but use them as the formatting/model example.

Sources



I am a freshman completing a Champion Document for a team game project.

My role is TL2+, and my assigned feature is “Summon System & Persistence.”

Your role in this conversation is limited to helping me create diagrams and format small tables. Do not write my introduction, scenarios, explanations, acceptance-test descriptions, conclusions, or AI-log summary unless I specifically request them later. I will write all narrative content myself.

Important rules:

1. Do not invent requirements, values, classes, interfaces, or behavior.
2. Use only the information I provide.
3. If necessary information is missing, ask me before creating the visual.
4. Keep every diagram simple enough for a freshman to explain.
5. Match the style of my uploaded samples: white background, dark navy outlines, peach highlighting for my feature, Arial-style font, and readable labels.
6. Make diagrams suitable for insertion into Microsoft Word.
7. Use correct UML notation for use-case diagrams.

8. Use correct Gane–Sarson notation for data-flow diagrams.
9. For every completed diagram, provide it as a downloadable PNG or SVG.
10. Do not add paragraphs or written sections around the diagram.

Feature information:

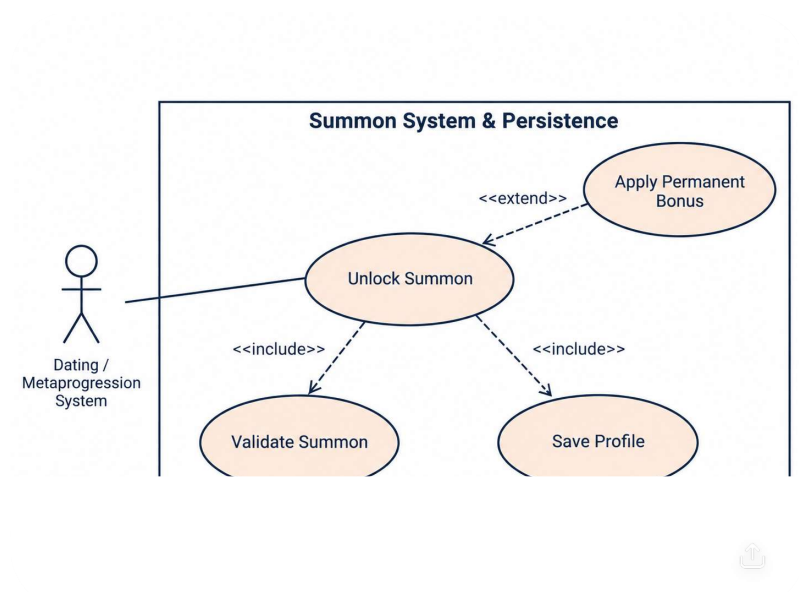
- A Summon is an AI-controlled helper that assists the player automatically.
- Summon subclasses are OrbitingSummon, HuntingSummon, TotemSummon, and RevengeSummon.
- Summon-related unlocks and permanent bonuses must remain saved after the game closes.
- TL1 owns Run State and Flow.
- TL2 owns Weapons and Attack resolution.
- TL2+ owns Summon behavior and summon-related persistence.
- TL3 owns Enemies.
- TL5 owns Dating and Metaprogression.
- TL6 owns Player Movement and Core Stats.
- My system may read player information and enemy targets and may send an attack request, but it does not calculate weapon damage.
- The Summon class hierarchy uses dynamic binding.
- Persistence uses the Memento pattern.
- Other team features communicate with the subsystem through SummonFacade.

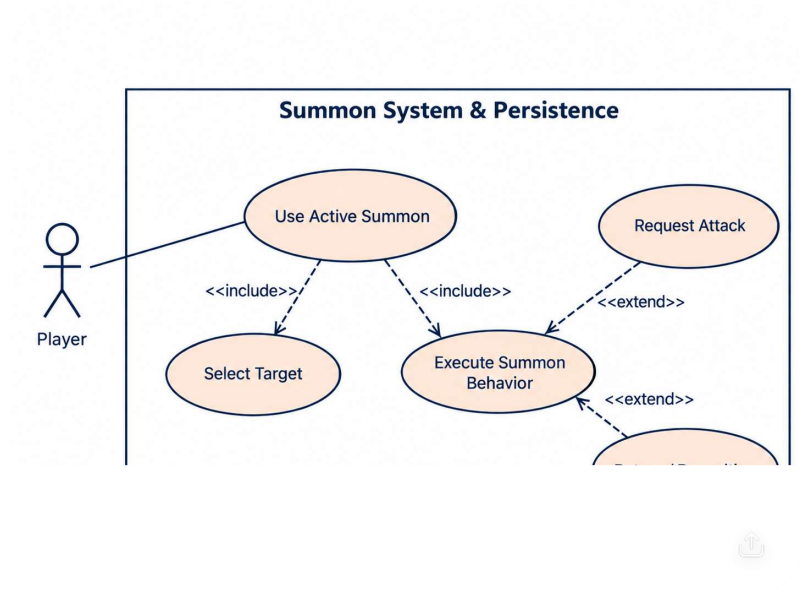
Wait for my first diagram request.

Show less ^

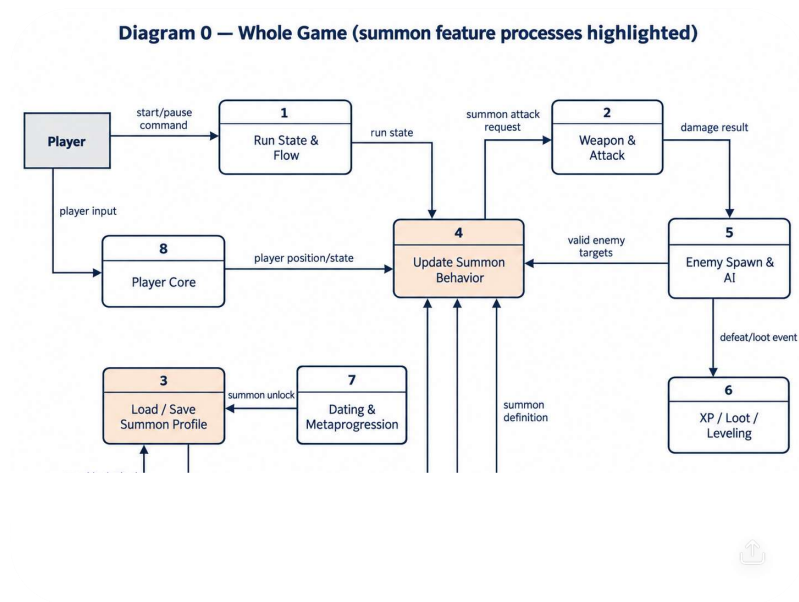


Worked for 51s



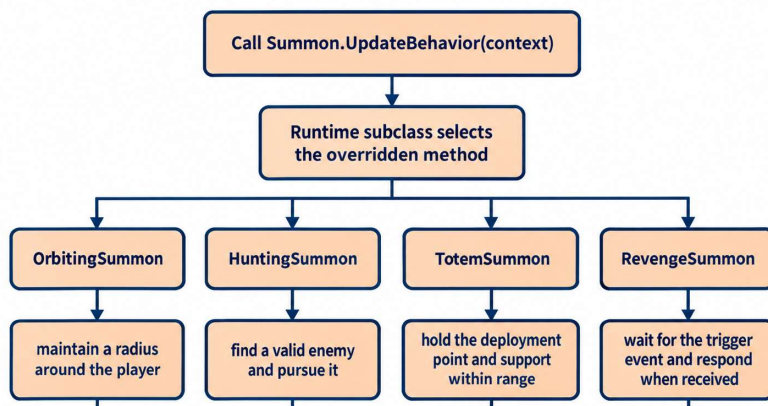


Worked for 1m 26s



Worked for 48s

Process Description — 4.2 Execute Dynamic Summon Behavior



Worked for 12s >

Using hour 0 as the project start, the calculations are:

Task	Work Item	ES	Duration	EF	LS	Slack
1	Summon data contracts and definitions	0	4	4	0	0
2	Base Summon hierarchy and Facade	4	6	10	4	0
3	Orbiting summon behavior	10	5	15	11	1

Task	Work Item	ES	Duration	EF	LS	Slack
4	Hunting summon behavior	10	6	16	10	0
5	Totem summon behavior	10	4	14	12	2
6	Revenge summon behavior	10	5	15	11	1
7	Profile persistence and Memento	4	7	11	9	5
8	Team-feature integration	16	6	22	16	0
9	Automated testing and debugging	22	6	28	22	0
10	Documentation and handoff	28	3	31	28	0

Total project duration: 31 hours

Critical path: 1 → 2 → 4 → 8 → 9 → 10

Tasks 1, 2, 4, 8, 9, and 10 have 0 slack, so they form the critical path.

I will not draw the PERT diagram until you confirm these calculations.

📄 ⬆️ ↺ 😊 🗨️ ...

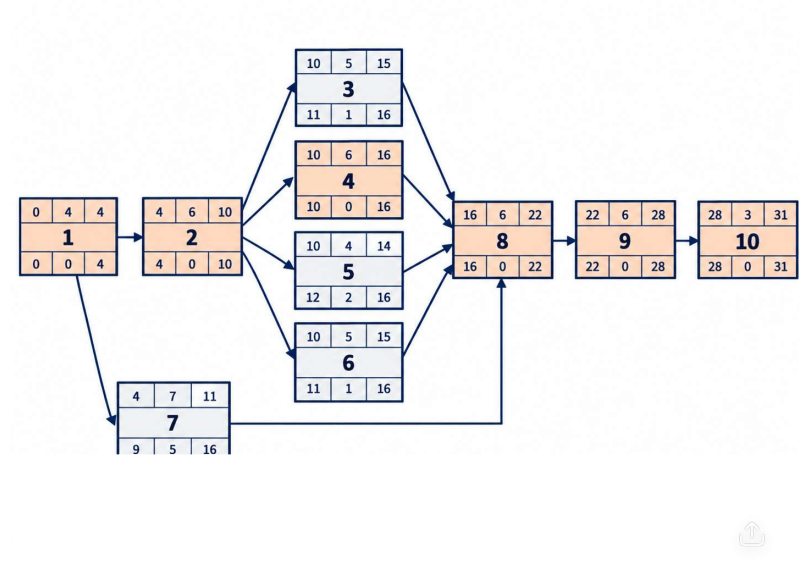
The PERT calculations are approved. Now create the downloadable PERT diagram.

Each node must show:

- Top row: ES | duration | EF
- Middle: task number
- Bottom row: LS | slack | LF

Highlight the critical-path nodes in peach. Clearly state the critical path and its total project hours below the diagram.

Worked for 53s



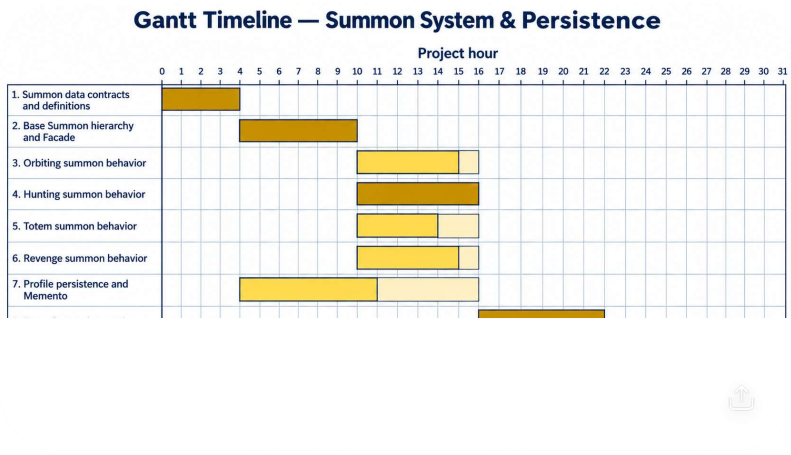
Using the approved PERT calculations and the same ten tasks, create a downloadable Gantt chart.

Requirements:

- Horizontal project-hour axis
- One row for each task
- Dark gold for critical-path work
- Lighter gold for noncritical work
- Pale color for available slack
- Preserve the exact task durations and predecessor relationships
- Include a small legend
- Do not change the approved schedule

Show less ^

Worked for 47s



ChatGPT can make mistakes. Check important info.